Software Requirements Specification

API Monitoring SaaS Platform

1. Introduction

1.1 Purpose

The purpose of this document is to outline the software requirements for the API Monitoring Software as a Service (SaaS) platform. This platform enables companies to register their APIs, assign them to employee accounts, monitor real-time uptime, response times, and key health statuses, and receive instant alerts when anomalies occur.

1.2 Scope

The SaaS application consists of a full-stack web interface with role-based access control, real-time background monitoring engines, and an analytics dashboard. It monitors APIs via HTTP checks, validates third-party API keys (e.g., OpenAI, Stripe), tracks slow load degradation, caches analytics, and sends email alerts upon failures.

2. Overall Description

2.1 Product Features

1. Multi-tenant Role-Based Access Control (RBAC): Company Admins can invite employees, manage subscriptions, and view all system endpoints. Employees can manage and view only their assigned endpoints.
2. API Endpoint Monitoring: Real-time polling to verify status codes, response times, and API key validities.
3. Third-Party API Key Validation: Integrations specifically detecting INVALID, LIMITED, and RATE_LIMITED HTTP signatures for external providers.
4. Performance Degradation Detection: Algorithmic identification of progressively slowing response times under load before a total timeout occurs.
5. Real-time Analytics Dashboard: Data visualization charts for Uptime, Average Response Times, and Error Rates.
6. Alert Notification System: SMTP email alerts dispatched to API owners when APIs go down or keys become invalid.
7. Background Cron Execution: Scheduled tasks running at specific intervals (e.g., every 60s) to perform the endpoint checks without blocking the main web server.

8. Automated Data Purging: Regular garbage collection of expired email verification OTPs and old endpoint monitoring logs to preserve database efficiency.

2.2 Operating Environment

- Backend: Python (Flask/Gunicorn)
- Database: PostgreSQL (Supabase/Neon)
- Frontend: HTML, CSS, JavaScript (Vanilla/Chart.js)
- Deployment: Vercel (Web Server), Independent Cron/Worker Threads for monitoring tasks.

3. Functional Requirements

3.1 Authentication & Authorization

- REQ-1: The system shall support secure registration via email OTP.
- REQ-2: The system shall authenticate users via JWT tokens.
- REQ-3: Company Admins must be able to invite Employee users.
- REQ-4: Employees shall only access and modify their own endpoints.
- REQ-5: Company Admins shall have read-only access to their employees' endpoints and aggregated statistical data.

3.2 API Endpoint Management

- REQ-6: Users shall be able to Create, Read, Update, and Delete (CRUD) API configurations (URL, Method, API Key Header, API Key).
- REQ-7: All stored API keys must be encrypted or masked during transmission and retrieval to prevent unauthorized exposure.

3.3 Monitoring Engine

- REQ-8: A background worker thread shall continuously execute HTTP calls based on each endpoint's defined interval.
- REQ-9: The engine shall log HTTP status codes, latency, and success booleans into a historical database table.
- REQ-10: Consecutive failures shall be tracked up to a customizable threshold before an alert is escalated.

3.4 Analytics & Reporting

- REQ-11: The system shall aggregate data from monitoring_logs to output a total Uptime Percentage per endpoint.
- REQ-12: The analytics engine shall use Redis/in-memory caching to optimize heavy aggregate queries

(e.g., P50/P95 latencies).

4. Non-Functional Requirements

4.1 Performance & Scalability

- NFR-1: Dashboard statistics must return within 200ms using caching mechanisms.
- NFR-2: The monitoring engine must parallelize HTTP checks using thread pools to ensure large endpoint numbers don't cause drift in polling intervals.

4.2 Security

- NFR-3: Passwords must be hashed using bcrypt algorithms.
- NFR-4: API requests shall utilize standard HTTPS.

4.3 Reliability & Availability

- NFR-5: The core Web API must isolate its PostgreSQL connection pooling to ensure it does not starve connection slots during serverless scaling events (using a custom DBWrapper).
- NFR-6: Multi-tenant degradation load test coverage must enforce SLA guarantees on response time spikes.